

Aline Scholl Santos – Matrícula: 202410008775

Ana Clara Maiberg – Matrícula: 202410004629

DOCUMENTAÇÃO DO PROJETO: HOSPITALAPP

1. Visão Geral do Projeto

O **HospitalApp** é uma aplicação em modo console desenvolvida com fins estritamente didáticos para simular o gerenciamento e fluxo de um sistema hospitalar. O software contempla as entidades fundamentais **Paciente**, **Médico** e **Consulta**.

2. Especificação Técnica e Tecnologias

- **Linguagem:** C# 12
- **Plataforma de Execução:** .NET 10.0
- **Tipo de Saída:** Console Application (Aplicativo de Console)
- **Dependências Externas:** Nenhuma (Uso exclusivo de bibliotecas nativas do SDK do .NET)

3. Padrões de Projeto (Design Patterns) Implementados

O núcleo do projeto baseia-se na implementação de quatro padrões de projeto do catálogo GoF (Gang of Four), divididos em suas respectivas categorias:

3.1. Singleton (Padrão Criacional)

- **Componente:** BancoDadosHospital
- **Finalidade:** Garantir a existência de uma, e apenas uma, instância de simulação do banco de dados em memória durante todo o ciclo de vida do processo.
- **Detalhes de Implementação:** Utiliza a classe nativa `Lazy<T>` do C# para garantir que a inicialização seja *lazy* (feita apenas quando necessária) e nativamente segura para concorrência de threads (*thread-safe*), eliminando a necessidade de blocos explícitos de travamento manual (lock).

3.2. Builder (Padrão Criacional)

- **Componente:** ConsultaBuilder
- **Finalidade:** Flexibilizar e organizar a criação do objeto complexo Consulta, permitindo uma construção passo a passo.
- **Detalhes de Implementação:** Oferece uma interface de programação fluente (*Fluent API*) por meio de encadeamento de métodos (*method chaining*). O objeto resultante só é exposto e instanciado após a invocação do método final

.Construir(), momento no qual as regras de validação obrigatórias do negócio são aplicadas.

3.3. Decorator (Padrão Estrutural)

- **Componentes:** DecoradorExames e DecoradorRetorno
- **Finalidade:** Adicionar responsabilidades, comportamentos e custos adicionais a uma consulta dinamicamente, sem a necessidade de modificar as classes base ou recorrer à herança excessiva.
- **Detalhes de Implementação:** Os decoradores envolvem o objeto base (ConsultaConcreta) em camadas sequenciais. Cada camada acrescenta metadados (como exames solicitados ou agendamento de retornos) e incrementa de forma cumulativa o valor financeiro total da consulta.

3.4. Observer (Padrão Comportamental)

- **Componentes:** SujeitoConsulta (Subject) e classes derivadas de IObservador.
- **Finalidade:** Definir uma relação de dependência "um para muitos", quando o estado de um agendamento muda (criação de consulta), todos os seus dependentes sejam notificados de forma automática.
- **Detalhes de Implementação:** Promove o desacoplamento, pois o SujeitoConsulta conhece apenas a interface genérica IObservador. No cenário implementado

4. Instruções para Implantação e Execução

4.1. Pré-requisitos do Sistema

- Possuir o **.NET 10 SDK** devidamente instalado na máquina de execução.
- Dispor de uma interface de linha de comando (Terminal, PowerShell ou Bash).

5. Logs de Saída do Sistema

Ao executar a aplicação através da classe Program.cs, o console imprimirá o fluxo unificado demonstrando o sucesso de execução de todas as implementações.